Precision-Weighted Score Composition for Hierarchical Simulation-Based Inference

Anonymous Authors

Draft — October 9, 2026

Abstract

1 Introduction

2 Background

2.1 Simulation-based inference

A simulator defines an implicit likelihood: given parameters $\theta \in \mathbb{R}^d$ we can sample $x \sim p(x \mid \theta)$, but we cannot evaluate $p(x \mid \theta)$. With a prior $p(\theta)$, the target is the posterior $p(\theta \mid x) \propto p(\theta)p(x \mid \theta)$ for an observed x. *Amortised* neural posterior estimation simulates pairs $(\theta, x) \sim p(\theta)p(x \mid \theta)$, trains a conditional density estimator $q_\psi(\theta \mid x)$ on them once, and then evaluates it on any observation without further simulation. In this paper the estimator is a conditional diffusion model, which represents the posterior through its score.

2.2 Score-based diffusion models for posteriors

Forward process. A diffusion model perturbs clean posterior draws $\theta_0 \sim p(\theta \mid x)$ with a Gaussian forward kernel

$$q_t(\theta_t \mid \theta_0) = \mathcal{N}(\theta_t; \alpha_t\theta_0, \sigma_t^2\mathbf{I}), \qquad t \in [0,1], \tag{1}$$

where α_t and σ_t are fixed schedules. For the variance-exploding (VE) SDE, $\alpha_t = 1$ and σ_t grows from ≈ 0 to a large $\sigma_{\max}$. For the variance-preserving (VP) SDE, $\alpha_t^2 + \sigma_t^2 = 1$. This yields a family of noised posteriors $p_t(\theta_t \mid x) = \int q_t(\theta_t \mid \theta_0)p(\theta_0 \mid x)\,\mathrm{d}\theta_0$ that interpolates between the posterior ($t = 0$) and an uninformative Gaussian ($t = 1$).

Variance-exploding coordinates. Every such process becomes variance exploding in the rescaled state $y = \theta_t/\alpha_t$ with noise level $\lambda_t = \sigma_t/\alpha_t$:

$$p_t(y \mid x) = \int \mathcal{N}(y; \theta_0, \lambda_t^2\mathbf{I})\, p(\theta_0 \mid x)\,\mathrm{d}\theta_0. \tag{2}$$

The score in y is α_t times the score in θ_t. We work in these coordinates throughout, write λ for λ_t when t is fixed, and write $s(y) = \nabla_y \log p_t(y \mid x)$ for the noised posterior score.

Reverse process. In y-coordinates the forward process is $\mathrm{d}y = \sqrt{\mathrm{d}\lambda_t^2/\mathrm{d}t}\,\mathrm{d}W_t$. It is reversed in time by

$$\mathrm{d}y = -\frac{\mathrm{d}\lambda_t^2}{\mathrm{d}t}\, s(y)\,\mathrm{d}t + \sqrt{\frac{\mathrm{d}\lambda_t^2}{\mathrm{d}t}}\,\mathrm{d}\bar{W}_t, \tag{3}$$

integrated from $t = 1$ to $t = 0$, or equivalently in distribution by the probability-flow ODE

$$\frac{\mathrm{d}y}{\mathrm{d}t} = -\frac{1}{2}\,\frac{\mathrm{d}\lambda_t^2}{\mathrm{d}t}\, s(y). \tag{4}$$

Both require only the score s at every noise level. Numerical solvers (Euler–Maruyama, DPM-Solver, Heun) discretise Equations (3) and (4). They can be combined with Langevin *corrector* steps $y \leftarrow y + \epsilon\, s(y) + \sqrt{2\epsilon}\, z$, $z \sim \mathcal{N}(0, \mathbf{I})$, which leave $p_t(\cdot \mid x)$ invariant when s is its exact score.

Training by denoising score matching. The score is learned with a network $s_\psi(y, t, x)$ that minimises

$$\mathbb{E}_{t,\,(\theta_0,x),\,\varepsilon}\left[w(t)\left\|s_\psi(\theta_0 + \lambda_t\varepsilon, t, x) + \frac{\varepsilon}{\lambda_t}\right\|^2\right], \tag{5}$$

with $\varepsilon \sim \mathcal{N}(0, \mathbf{I})$, (θ_0, x) drawn from the simulator and a positive weighting $w(t)$. The minimiser is the noised posterior score: $s_\psi(y, t, x) = \nabla_y \log p_t(y \mid x)$.

Backward kernel and Tweedie's identities. For fixed y, the *backward kernel* is the conditional law of the clean parameter given the noisy one:

$$\pi_t(\theta_0 \mid y, x) = \frac{\mathcal{N}(y; \theta_0, \lambda^2\mathbf{I})\, p(\theta_0 \mid x)}{p_t(y \mid x)}. \tag{6}$$

Tweedie's formula ties its mean to the score. The second-order identity ties its covariance to the score Jacobian:

$$s(y) = \frac{\mathbb{E}_{\pi_t}[\theta_0 \mid y, x] - y}{\lambda^2}, \tag{7}$$

$$\nabla_y s(y) = \frac{\mathrm{Cov}_{\pi_t}[\theta_0 \mid y, x] - \lambda^2\mathbf{I}}{\lambda^4}. \tag{8}$$

Both identities hold for any posterior, not only Gaussian ones (Sections 3.2 and 3.7 re-derive them). They are the tools for everything that follows: Equation (7) converts between scores and denoised means, and Equation (8) between score Jacobians and denoising uncertainty.

2.3 Tall data and compositional score modelling

Suppose n observations $\mathbf{x}_{1:n}$ are conditionally independent given one shared θ. Bayes' rule gives the *tall-data* factorisation

$$p(\theta \mid \mathbf{x}_{1:n}) \;\propto\; p(\theta)^{1-n} \prod_{j=1}^{n} p(\theta \mid x_j). \tag{9}$$

Training a posterior estimator directly on sets of n observations requires simulating at that n, and it has to be repeated for every n. Compositional score modelling instead trains a single-observation network $s_\psi(y, t, x_j) \approx s_j(y) := \nabla_y \log p_t(y \mid x_j)$ once. It then builds the score of the tall posterior from the n single-observation scores at inference time.

The obstacle. Equation (9) is a statement about clean densities. Diffusion convolves the posterior with the forward kernel, and convolution does not commute with products:

$$p_t(y \mid \mathbf{x}_{1:n}) \;\not\propto\; p_t(y)^{1-n} \prod_{j=1}^{n} p_t(y \mid x_j) \qquad \text{for } t > 0. \tag{10}$$

The right-hand side of Equation (10) is exact only at $t = 0$. Every compositional method is a way of handling this mismatch.

2.4 Competing composition rules for a shared parameter

Uncorrected sum. The simplest rule takes the score of the right-hand side of Equation (10),

$$s_{\text{sum}}(y) = (1-n)\, s_0(y) + \sum_{j=1}^{n} s_j(y), \tag{11}$$

where $s_0(y) = \nabla_y \log p_t(y)$ is the diffused prior score. It is the correct tall-data score at $t = 0$ and wrong at every intermediate noise level. The reverse process then follows the wrong path of distributions, and the error grows with n.

F-NPSE: annealed Langevin. Geffner et al. [2] use the bridge

$$s_{\text{F}}(y, t) = (1-n)(1-t)\, \nabla_y \log p(y) + \sum_{j=1}^{n} s_j(y), \tag{12}$$

with the *un*diffused prior score, annealed by $(1-t)$. Equation (12) is not the score of the noised tall posterior. They therefore do not integrate Equation (3). Instead they run unadjusted Langevin dynamics $y \leftarrow y + \epsilon_t s_{\text{F}}(y, t) + \sqrt{2\epsilon_t} z$ for several steps at each noise level of a decreasing schedule, so that MCMC corrects the mismatch. The cost is many score evaluations per noise level, and the step sizes are sensitive tuning parameters. For a VE diffusion and large n, the bridge densities defined by Equation (12) can become improper at intermediate t (negative total precision), which makes the Langevin chains diverge. Our implementation therefore uses the diffused prior score in Equation (12).

Gauss and Jac: precision-weighted composition. Linhart et al. [6] start from an identity that holds exactly. Writing $\pi_t^0(\theta_0 \mid y)$ for the backward kernel of the prior and $\pi_t(\theta_0 \mid y, x_j)$ for those of the single-observation posteriors, and splitting the forward kernel so that each factor of Equation (9) receives one copy, gives

$$p_t(y \mid \mathbf{x}_{1:n}) \propto p_t(y)^{1-n} \prod_j p_t(y \mid x_j) \times \underbrace{\int \pi_t^0(\theta_0 \mid y)^{1-n} \prod_j \pi_t(\theta_0 \mid y, x_j)\, \mathrm{d}\theta_0}_{=: Z_t(y)}. \tag{13}$$

The tall-data score is therefore the uncorrected sum Equation (11) plus a correction $\nabla_y \log Z_t(y)$. To evaluate the correction they approximate every backward kernel by a Gaussian, with means given by Equation (7) and covariances $\Sigma_{t,j}$ (observations) and $\Sigma_{t,0}$ (prior). Differentiating the resulting closed form gives a leading term

$$\nabla_y \log p_t(y \mid \mathbf{x}_{1:n}) \approx \Lambda^{-1} \Big[\textstyle\sum_j \Sigma_{t,j}^{-1} s_j + (1-n)\, \Sigma_{t,0}^{-1} s_0 \Big], \tag{14}$$

with $\Lambda = \sum_j \Sigma_{t,j}^{-1} + (1-n)\, \Sigma_{t,0}^{-1}$, plus a residual that vanishes when the covariances do not depend on y. Equation (14) is a precision-weighted average. An observation whose backward kernel is narrow, i.e. one that is informative at this noise level, receives a large weight. Because the rule is a valid approximation of the noised tall score at every t, it can be used inside the ordinary reverse process without Langevin steps. Two estimators of $\Sigma_{t,j}$ are proposed:

- Gauss assumes that every single-observation posterior is Gaussian, $p(\theta \mid x_j) = \mathcal{N}(\mu_j, \Sigma_{0,j})$. For a Gaussian, the backward kernel has the closed-form precision $\Sigma_{t,j}^{-1} = \Sigma_{0,j}^{-1} + \lambda^{-2}\mathbf{I}$ (product of two Gaussians in θ_0). $\Sigma_{0,j}$ is the empirical covariance of a *pilot* run: one full reverse diffusion per observation, with the same network, before inference starts. The prior uses $\Sigma_{t,0}^{-1} = \Sigma_{\text{pr}}^{-1} + \lambda^{-2}\mathbf{I}$ for a Gaussian prior $\mathcal{N}(\mu_{\text{pr}}, \Sigma_{\text{pr}})$.
- Jac uses Equation (8) instead, $\Sigma_{t,j}^{-1} = \lambda^{-2}\big(\mathbf{I} + \lambda^2 \nabla_y s_\psi(y, t, x_j)\big)^{-1}$. This needs no Gaussian assumption and no pilot, but it does need a Jacobian of the network at every step. Its stability is identified as an open problem, since a network Jacobian need not yield a positive-definite covariance.

Both rules are derived for one parameter block that every observation shares. In practice, Λ can become indefinite for large n, when an estimated observation precision is smaller than the prior precision in some direction.

Damped sum. Arruda et al. [1] keep the plain sum but rescale it with a time-dependent damping factor $d(t)$, optionally using a random mini-batch B of $m \le n$ observations per step. In the form we implement (their Eq. 9),

$$s_{\text{damp}}(y,t) = d(t)\Big[(1-n)(1-t)\, s_0(y) + \frac{n}{m}\sum_{j\in B} s_j(y)\Big], \tag{15}$$

with d decreasing from $d(0) \approx 1$ at the data end to a small value at the noise end; our default interpolates exponentially down to $d(1) = n^{-1/2}$. Damping suppresses the n-fold growth of the score at high noise, where the sum is most wrong, so the reverse process stays stable. It does not correct the bridge: Equation (15) is not an approximation of the noised tall score, and its accuracy depends on the schedule.

Structured tall data. Gloeckler et al. [4] apply Gauss/Jac-type corrections to Markovian time series, where the factors are overlapping windows of one trajectory. The composed parameter is still shared by all factors.

2.5 Hierarchical models

In a hierarchical model, a *global* parameter $g \in \mathbb{R}^{d_g}$ is shared by n groups (subjects, trials, stars, ...). Every group j also has a *local* parameter $\ell_j \in \mathbb{R}^{d_\ell}$ and an observation x_j:

$$p(g, \ell_{1:n}, \mathbf{x}_{1:n}) = p(g)\prod_{j=1}^{n} p(\ell_j \mid g)\, p(x_j \mid g, \ell_j). \tag{16}$$

The local prior may depend on g; partial pooling is the typical case, where g contains a population mean and scale of the locals. We write $\vartheta_j = (g, \ell_j) \in \mathbb{R}^D$, $D = d_g + d_\ell$, for the block that observation j sees, and $\theta = (g, \ell_1, \dots, \ell_n)$ for the full latent vector, of dimension $d_g + n d_\ell$. The posterior dimension therefore grows with n, while every simulation only ever involves one group. A compositional model is trained on single groups ($n = 1$). For every j it returns the *joint* score of the single-group posterior $p(\vartheta_j \mid x_j)$:

$$s_j(y_j) = \big(s_j^g, s_j^\ell\big) \approx \nabla_{y_j} \log p_t(y_j \mid x_j), \qquad y_j = (g_t, \ell_{j,t}). \tag{17}$$

Why flat composition does not apply. All rules of Section 2.4 compose the scores of a block that every observation shares. Here only g is shared. Composing g alone with Equation (14) would require the *marginal* score $\nabla_{g_t} \log p_t(g_t \mid x_j)$. The network instead provides $s_j^g = \partial_{g_t} \log p_t(g_t, \ell_{j,t} \mid x_j)$, which is *conditional* on the current local state. The two are related by

$$\nabla_{g_t} \log p_t(g_t \mid x_j) = \mathbb{E}\big[s_j^g \mid g_t, x_j\big], \tag{18}$$

an expectation over $\ell_{j,t}$ that is not available at sampling time.

2.6 Competing approaches for hierarchical models

Joint amortised estimation. One can train an estimator on whole hierarchical data sets, e.g. with an exchangeable set encoder over groups. This approach is taken by HNPE [8], by amortised multilevel models [5], and by all-in-one transformers that condition on every group [3]. These methods simulate entire data sets during training, so their training cost grows with n. Generalising to n outside the training range is not guaranteed.

Plain Gauss on the global block. A direct heuristic plugs the network's conditional score s_j^g into Equation (14) with the marginal pilot precision $([\Sigma_{0,j}]_{gg})^{-1} + \lambda^{-2}\mathbf{I}$, and leaves the local scores s_j^ℓ as they are. This uses a weight that belongs to the marginal score on the conditional score, and it ignores how the locals should respond when composition moves g. It is biased even with exact scores (Section 5).

Two-stage compositional sampling. Arruda et al. [1] split the problem. A global network models $p(g \mid x_j)$, the marginal over the locals, and its scores are composed with the damped sum Equation (15). A second network models $p(\ell_j \mid g, x_j)$ and samples every local conditionally on the sampled global. This sidesteps Equation (18) by learning the marginal directly. It still inherits the uncorrected bridge of Equation (15), and it needs two networks.

Blockwise Gibbs inside the diffusion. Zaballa and Hui [9] alternate block updates inside the reverse SDE. Each local is updated given the current global, and the global given all locals with an F-NPSE-type sum, over several re-noised rounds and with an n-dependent diffusion coefficient. This scheme has no precision weighting and does not marginalise the locals.

Summary of the gap. Precision-weighted rules are the only corrections that approximate the noised tall score at every t, but they exist only for a shared block. Hierarchical compositional methods use uncorrected or damped sums. Section 3 derives the precision-weighted rule for the hierarchical case, using only the conditional scores that a single joint network provides.

3 Methods

We derive one composition rule for the hierarchical model Equation (16) and then obtain two methods from it. The derivation has four steps. First, the tall-data backward kernel factorises *exactly* into the single-observation backward kernels (Section 3.1). Second, Tweedie's formula turns the composed score into the mean of that kernel (Section 3.2). Third, a single approximation replaces each factor by a Gaussian, which turns the composed score into the solution of an arrowhead linear system (Section 3.3). Fourth, the local blocks of that system are eliminated in closed form (Section 3.4). The two methods differ only in where the per-observation covariances come from. `gauss_hierarchical` estimates them once from a pilot run (Section 3.6); `gauss_jacobian` reads them from the network's Jacobian at the current state (Section 3.7).

Notation. Fix a time t and write $\lambda = \lambda_t$. The full noisy state is $y = (g_t, \ell_{1,t}, \dots, \ell_{n,t})$, and observation j sees the sub-vector $y_j = (g_t, \ell_{j,t}) \in \mathbb{R}^D$. Clean quantities carry a subscript 0: $\theta_0 = (g_0, \ell_{1,0}, \dots, \ell_{n,0})$ and $\vartheta_{j,0} = (g_0, \ell_{j,0})$. E_j is the selection matrix with $E_j\theta = \vartheta_j$, and E_g is the one with $E_g\theta = g$. $\varphi_\lambda(z) = \mathcal{N}(z; 0, \lambda^2\mathbf{I})$ is the isotropic Gaussian density in whatever dimension z has. Since φ_λ factorises over coordinates,

$$\varphi_\lambda(y - \theta_0) = \varphi_\lambda(g_t - g_0) \prod_{j=1}^{n} \varphi_\lambda(\ell_{j,t} - \ell_{j,0}). \tag{19}$$

3.1 Step 1: The backward kernel factorises exactly

We first need the hierarchical version of the tall-data factorisation Equation (9). Bayes' rule for a single group gives $p(\vartheta_j \mid x_j) = p(g)\, p(\ell_j \mid g)\, p(x_j \mid g, \ell_j)/p(x_j)$. Taking the product over all groups,

$$\prod_{j=1}^{n} p(\vartheta_j \mid x_j) = \frac{p(g)^n}{\prod_j p(x_j)} \prod_{j=1}^{n} p(\ell_j \mid g)\, p(x_j \mid g, \ell_j).$$

Multiplying by $p(g)^{1-n}$ leaves $p(g)\prod_j p(\ell_j \mid g) p(x_j \mid g, \ell_j)$, which is the joint Equation (16) and therefore proportional in θ to the posterior:

$$p(\theta \mid \mathbf{x}_{1:n}) \;\propto\; p(g)^{1-n} \prod_{j=1}^{n} p(\vartheta_j \mid x_j). \tag{20}$$

Each local prior $p(\ell_j \mid g)$ appears in exactly one factor, so only the global prior is over-counted and only it needs the correction $p(g)^{1-n}$.

The tall-data posterior is diffused exactly like Equation (2), $p_t(y \mid \mathbf{x}_{1:n}) = \int \varphi_\lambda(y - \theta_0)\, p(\theta_0 \mid \mathbf{x}_{1:n})\, \mathrm{d}\theta_0$, and its backward kernel is

$$\pi_t(\theta_0 \mid y, \mathbf{x}_{1:n}) = \frac{\varphi_\lambda(y - \theta_0)\, p(\theta_0 \mid \mathbf{x}_{1:n})}{p_t(y \mid \mathbf{x}_{1:n})}. \tag{21}$$

The single-observation backward kernels are $\pi_t(\vartheta_{j,0} \mid y_j, x_j) \propto \varphi_\lambda(y_j - \vartheta_{j,0})\, p(\vartheta_{j,0} \mid x_j)$, and the backward kernel of the global prior is $\pi_t^0(g_0 \mid g_t) \propto \varphi_\lambda(g_t - g_0)\, p(g_0)$.

To relate the tall kernel to these, keep y and $\mathbf{x}_{1:n}$ fixed and read all proportionalities in θ_0. Inserting Equation (20) into Equation (21) and factorising the forward kernel with Equation (19) gives

$$\pi_t(\theta_0 \mid y, \mathbf{x}_{1:n}) \propto p(g_0)^{1-n}\, \varphi_\lambda(g_t - g_0) \times \prod_{j=1}^{n} \Big[p(\vartheta_{j,0} \mid x_j)\, \varphi_\lambda(\ell_{j,t} - \ell_{j,0}) \Big].$$

The global noise factor appears only once, but each of the n single-observation kernels needs a copy of it. We therefore split it as $\varphi_\lambda(g_t - g_0) = \varphi_\lambda(g_t - g_0)^{1-n} \cdot \varphi_\lambda(g_t - g_0)^n$ and hand one copy of the second power to each j:

$$\pi_t(\theta_0 \mid y, \mathbf{x}_{1:n}) \propto \Big[p(g_0) \varphi_\lambda(g_t - g_0) \Big]^{1-n} \times \prod_{j=1}^{n} \Big[p(\vartheta_{j,0} \mid x_j)\, \varphi_\lambda(g_t - g_0)\, \varphi_\lambda(\ell_{j,t} - \ell_{j,0}) \Big].$$

By Equation (19) applied to the D-dimensional block, the j-th bracket equals $p(\vartheta_{j,0} \mid x_j)\, \varphi_\lambda(y_j - \vartheta_{j,0}) = \pi_t(\vartheta_{j,0} \mid y_j, x_j)\, p_t(y_j \mid x_j)$. The first bracket equals $\pi_t^0(g_0 \mid g_t)\, p_t(g_t)$. The factors $p_t(y_j \mid x_j)$ and $p_t(g_t)^{1-n}$ do not depend on θ_0 and go into the normaliser $Z_t(y)$, which leaves

$$\pi_t(\theta_0 \mid y, \mathbf{x}_{1:n}) = \frac{\pi_t^0(g_0 \mid g_t)^{1-n}}{Z_t(y)} \prod_{j=1}^{n} \pi_t(\vartheta_{j,0} \mid y_j, x_j). \tag{22}$$

Equation (22) involves no approximation. The tall-data backward kernel is a product of exactly the single-observation backward kernels that the trained network describes, together with a power of the prior kernel. This is why composition can work at all: Equation (9) fails after diffusion at the level of marginal densities (Equation (10)), but it survives at the level of backward kernels.

3.2 Step 2: The composed score is a backward-kernel mean

Tweedie's formula Equation (7) holds for any distribution p with a finite first moment. To see this, let $p_t(y) = \int \varphi_\lambda(y - \theta_0)\, p(\theta_0)\, \mathrm{d}\theta_0$ and $\pi_t(\theta_0 \mid y) \propto \varphi_\lambda(y - \theta_0) p(\theta_0)$. Since $\nabla_y \varphi_\lambda(y - \theta_0) = \varphi_\lambda(y - \theta_0)\, (\theta_0 - y)/\lambda^2$, differentiating under the integral sign gives $\nabla_y p_t(y) = \int \varphi_\lambda(y - \theta_0)\, \frac{\theta_0 - y}{\lambda^2}\, p(\theta_0)\, \mathrm{d}\theta_0$. Differentiation under the integral is allowed because the Gaussian kernel and its derivative are bounded (dominated convergence). Dividing by $p_t(y)$ turns the integral into an expectation under $\pi_t(\cdot \mid y)$:

$$\nabla_y \log p_t(y) = \frac{\mathbb{E}_{\pi_t}[\theta_0 \mid y] - y}{\lambda^2}. \tag{23}$$

Applying Equation (23) to the tall-data posterior on the full latent vector θ, the composed score $u(y) :=$

$\nabla_y \log p_t(y \mid \mathbf{x}_{1:n})$, which has a global block and one local block per observation, is

$$u(y) = \frac{\mu(y) - y}{\lambda^2}, \qquad \mu(y) := \mathbb{E}_{\pi_t}[\theta_0 \mid y, \mathbf{x}_{1:n}]. \tag{24}$$

Equation (24) is exact. Computing the composed score at y is therefore the same task as computing the mean of the composed backward kernel Equation (22) at that y. Nothing in this reduction differentiates the backward kernel with respect to y.

Applied to each single-observation posterior and to the global prior, Equation (23) also gives the means of their backward kernels in terms of scores:

$$\begin{aligned} m_j &:= \mathbb{E}\big[\vartheta_{j,0} \mid y_j, x_j\big] = y_j + \lambda^2 s_j(y_j), \\ m_0 &:= \mathbb{E}\big[g_0 \mid g_t\big] = g_t + \lambda^2 s_0(g_t), \end{aligned} \tag{25}$$

where $s_j = (s_j^g, s_j^\ell)$ is the joint row score Equation (17) and $s_0(g_t) = \nabla_{g_t} \log p_t(g_t)$ is the diffused global prior score.

3.3 Step 3: Gaussian factors give an arrowhead system

The method makes exactly one approximation. At the current (t, y), we replace every factor in Equation (22) by a Gaussian with the correct mean Equation (25):

$$\begin{aligned} \pi_t(\vartheta_{j,0} \mid y_j, x_j) &\approx \mathcal{N}\big(\vartheta_{j,0};\, m_j,\, \Sigma_{t,j}\big), \\ \pi_t^0(g_0 \mid g_t) &\approx \mathcal{N}\big(g_0;\, m_0,\, \Sigma_{t,0}\big), \end{aligned} \tag{26}$$

with symmetric positive-definite covariances $\Sigma_{t,j} \in \mathbb{R}^{D\times D}$ and $\Sigma_{t,0} \in \mathbb{R}^{d_g \times d_g}$. We write $P_j := \Sigma_{t,j}^{-1}$ and $P_0 := \Sigma_{t,0}^{-1}$. The covariances may depend on y: everything below happens at one fixed y and never differentiates them. For a Gaussian prior $p(g) = \mathcal{N}(\mu_{\mathrm{pr}}, \Sigma_{\mathrm{pr}})$, the prior factor is exactly Gaussian, with $P_0 = \Sigma_{\mathrm{pr}}^{-1} + \lambda^{-2}\mathbf{I}$ (Equation (40) below). We split each per-observation precision into global and local blocks,

$$P_j = \begin{pmatrix} A_j & B_j \\ B_j^\top & C_j \end{pmatrix}, \tag{27}$$

with $A_j \in \mathbb{R}^{d_g \times d_g}$, $B_j \in \mathbb{R}^{d_g \times d_\ell}$ and $C_j \in \mathbb{R}^{d_\ell \times d_\ell}$.

Product of the Gaussians. With Equation (26) in place, the logarithm of the surrogate composed kernel is, up to a constant,

$$\begin{aligned} &-\tfrac{1-n}{2}(E_g\theta_0 - m_0)^\top P_0 (E_g\theta_0 - m_0) \\ &\qquad -\tfrac{1}{2}\textstyle\sum_j (E_j\theta_0 - m_j)^\top P_j (E_j\theta_0 - m_j). \end{aligned}$$

For $n \geq 2$ the factor $1 - n$ is negative, so the first term is a quadratic with a negative-definite "precision". This does no harm: the whole expression is still a quadratic form in θ_0, and expanding it gives $-\frac{1}{2}\theta_0^\top \Lambda \theta_0 + \theta_0^\top b + \text{const}$ with

$$\begin{aligned} \Lambda &:= (1-n)\, E_g^\top P_0 E_g + \textstyle\sum_{j=1}^n E_j^\top P_j E_j, \\ b &:= (1-n) E_g^\top P_0 m_0 + \textstyle\sum_j E_j^\top P_j m_j. \end{aligned} \tag{28}$$

As long as $\Lambda \succ 0$, completing the square shows that the surrogate is the Gaussian $\mathcal{N}(\theta_0; \Lambda^{-1}b, \Lambda^{-1})$, with mean $\mu = \Lambda^{-1}b$.

From means back to scores. Inserting this mean into Equation (24) gives $u = (\Lambda^{-1}b - y)/\lambda^2 = \Lambda^{-1}(b - \Lambda y)/\lambda^2$. Because $E_g y = g_t$ and $E_j y = y_j$,

$$\Lambda y = (1-n) E_g^\top P_0 g_t + \textstyle\sum_j E_j^\top P_j y_j.$$

Subtracting this from b and using Equation (25), i.e. $m_0 - g_t = \lambda^2 s_0$ and $m_j - y_j = \lambda^2 s_j$,

$$\begin{aligned} b - \Lambda y &= (1-n) E_g^\top P_0 (m_0 - g_t) + \textstyle\sum_j E_j^\top P_j (m_j - y_j) \\ &= \lambda^2\, r, \end{aligned}$$

where

$$r := (1-n)\, E_g^\top P_0\, s_0(g_t) + \textstyle\sum_{j=1}^n E_j^\top P_j\, s_j(y_j). \tag{29}$$

The factors of λ^2 cancel, and the composed score solves

$$\Lambda\, u = r. \tag{30}$$

The noisy state y has dropped out exactly, because each mean and each state carries the same precision in b and Λy.

Arrowhead structure. $E_j^\top P_j E_j$ places A_j in the (g, g) block, B_j in (g, ℓ_j), $B_j^\top$ in (ℓ_j, g), C_j in (ℓ_j, ℓ_j), and zeros elsewhere. $E_g^\top P_0 E_g$ touches only (g, g). No term couples ℓ_j with ℓ_k for $j \neq k$, so Λ is an *arrowhead* matrix,

$$\Lambda = \begin{pmatrix} M & B_1 & \cdots & B_n \\ B_1^\top & C_1 & & 0 \\ \vdots & & \ddots & \\ B_n^\top & 0 & & C_n \end{pmatrix}, \quad M := (1-n)P_0 + \sum_j A_j, \tag{31}$$

and, with the same placement applied to $P_j s_j = (A_j s_j^g + B_j s_j^\ell,\ B_j^\top s_j^g + C_j s_j^\ell)$, the right-hand side has the blocks

$$\begin{aligned} r^g &= (1-n)P_0 s_0 + \textstyle\sum_j \big(A_j s_j^g + B_j s_j^\ell\big), \\ r^{\ell_j} &= B_j^\top s_j^g + C_j s_j^\ell. \end{aligned} \tag{32}$$

Equation (30) has dimension $d_g + n d_\ell$. Its arrowhead structure lets us solve it in closed form at a cost linear in n.

3.4 Step 4: Eliminating the local blocks

Two block-inverse identities. We need two facts about a positive-definite $P = \left(\begin{smallmatrix} A & B \\ B^\top & C \end{smallmatrix}\right)$ with inverse $\Sigma = P^{-1} = \left(\begin{smallmatrix} \Sigma_{gg} & \Sigma_{g\ell} \\ \Sigma_{\ell g} & \Sigma_{\ell\ell} \end{smallmatrix}\right)$. C is a principal sub-matrix of P and hence positive definite. The Schur complement $S := A - BC^{-1}B^\top$ is positive definite too: for $v \neq 0$, setting $w = (v, -C^{-1}B^\top v)$ gives $w^\top P w = v^\top S v > 0$. The first block column of Σ is $(S^{-1}, -C^{-1}B^\top S^{-1})$, as a direct multiplication shows:

$$P\begin{pmatrix} S^{-1} \\ -C^{-1}B^\top S^{-1} \end{pmatrix} = \begin{pmatrix} (A - BC^{-1}B^\top)S^{-1} \\ B^\top S^{-1} - B^\top S^{-1} \end{pmatrix} = \begin{pmatrix} \mathbf{I} \\ 0 \end{pmatrix}.$$

Hence $\Sigma_{gg} = S^{-1}$ and $\Sigma_{\ell g} = -C^{-1}B^\top S^{-1}$. Right-multiplying the second identity by $\Sigma_{gg}^{-1} = S$ gives

$$S = \Sigma_{gg}^{-1}, \qquad -C^{-1}B^\top = \Sigma_{\ell g}\,\Sigma_{gg}^{-1}. \tag{33}$$

For every observation we accordingly define the *effective global precision* and the *local regression matrix*

$$\begin{aligned} S_j &:= A_j - B_j C_j^{-1} B_j^\top = \left([\Sigma_{t,j}]_{gg}\right)^{-1}, \\ R_j &:= -C_j^{-1} B_j^\top = [\Sigma_{t,j}]_{\ell g}\,[\Sigma_{t,j}]_{gg}^{-1}. \end{aligned} \tag{34}$$

S_j is the precision of the g-marginal of observation j's Gaussian backward kernel. R_j is the coefficient of the linear regression of $\ell_{j,0}$ on g_0 under that kernel.

Local rows. By Equations (31) and (32), the row block of ℓ_j in $\Lambda u = r$ reads

$$B_j^\top u^g + C_j u^{\ell_j} = B_j^\top s_j^g + C_j s_j^\ell.$$

C_j is invertible, so

$$\begin{aligned} u^{\ell_j} &= s_j^\ell + C_j^{-1} B_j^\top \left(s_j^g - u^g\right) \\ &= s_j^\ell - C_j^{-1} B_j^\top \left(u^g - s_j^g\right). \end{aligned} \tag{35}$$

Global row. The row block of g reads

$$\begin{aligned} &\Big[(1-n)P_0 + \textstyle\sum_j A_j\Big] u^g + \textstyle\sum_j B_j u^{\ell_j} \\ &\quad = (1-n)P_0 s_0 + \textstyle\sum_j \left(A_j s_j^g + B_j s_j^\ell\right). \end{aligned}$$

Substituting Equation (35) for the locals gives

$$\sum_j B_j u^{\ell_j} = \sum_j B_j s_j^\ell + \sum_j B_j C_j^{-1} B_j^\top \left(s_j^g - u^g\right).$$

The term $\sum_j B_j s_j^\ell$ now appears on both sides and cancels. Collecting the terms in u^g on the left and those in s_j^g on the right,

$$\begin{aligned} &\Big[(1-n)P_0 + \textstyle\sum_j \left(A_j - B_j C_j^{-1} B_j^\top\right)\Big] u^g \\ &\quad = (1-n)P_0 s_0 + \textstyle\sum_j \left(A_j - B_j C_j^{-1} B_j^\top\right) s_j^g, \end{aligned}$$

that is, $\Lambda_S\, u^g = (1-n)P_0 s_0 + \sum_j S_j s_j^g$ with the *composed global precision*

$$\Lambda_S := \sum_{j=1}^n S_j + (1-n)P_0. \tag{36}$$

When the system is solvable. Write $\Lambda = \left(\begin{smallmatrix} M & \mathbf{B} \\ \mathbf{B}^\top & \mathbf{C} \end{smallmatrix}\right)$ with $\mathbf{B} = (B_1, \ldots, B_n)$ and $\mathbf{C} = \mathrm{diag}(C_1, \ldots, C_n) \succ 0$. By the same argument as for S above, a symmetric block matrix with $\mathbf{C} \succ 0$ is positive definite if and only if its Schur complement $M - \mathbf{B}\mathbf{C}^{-1}\mathbf{B}^\top$ is. Because $\mathbf{C}$ is block diagonal, $\mathbf{B}\mathbf{C}^{-1}\mathbf{B}^\top = \sum_j B_j C_j^{-1} B_j^\top$, so this Schur complement is exactly Λ_S. Hence $\Lambda \succ 0$ if and only if $\Lambda_S \succ 0$, and in that case the solution is unique.

The hierarchical composition rule. Putting the two rows together, the composed score is

$$u^g = \Lambda_S^{-1}\Big[\textstyle\sum_{j=1}^n S_j\, s_j^g + (1-n)\, P_0\, s_0(g_t)\Big], \tag{37}$$

$$u^{\ell_j} = s_j^\ell + R_j\big(u^g - s_j^g\big), \quad j = 1, \ldots, n. \tag{38}$$

Three features of this rule matter in practice. First, the global rule Equation (37) uses only the network's *conditional* global scores s_j^g. The local scores s_j^ℓ cancelled exactly, so the unavailable marginal score Equation (18) is never needed. The correct weight for a conditional score is the effective precision S_j, and the locals pay for it through Equation (38). Second, the local rule has a direct reading in terms of denoised estimates. Multiplying Equation (38) by λ^2 and using Equation (25), the composed denoised local estimate departs from observation j's own estimate by R_j times the departure of the composed global estimate. This is the Gaussian conditional-mean update of $\ell_{j,0}$ given a revised g_0. Third, the cost is linear in n: n inversions of $d_\ell \times d_\ell$ blocks and one $d_g \times d_g$ solve.

3.5 Properties of the rule

Consistency checks. For one observation ($n = 1$) the prior term has coefficient $1-n = 0$, so $u^g = S_1^{-1} S_1 s_1^g = s_1^g$ and $u^{\ell_1} = s_1^\ell$: the rule returns the network score unchanged. Without locals ($d_\ell = 0$), B_j and C_j are empty, $S_j = P_j$, and Equation (37) is the flat rule Equation (14). Without coupling ($[\Sigma_{t,j}]_{\ell g} = 0$, which forces $B_j = 0$), Equation (34) gives $R_j = 0$ and $S_j = A_j$: the locals keep their network scores and g is composed with the conditional precision.

Information form. Define the information that observation j carries about g as $I_j := S_j - P_0$. Since $\sum_j S_j + (1-n)P_0 = \sum_j (S_j - P_0) + nP_0 + (1-n)P_0$,

$$\Lambda_S = P_0 + \sum_{j=1}^n I_j. \tag{39}$$

In particular, $\Lambda_S \succeq P_0 \succ 0$ whenever every $I_j \succeq 0$. This is the basis of the safeguard in Section 4.7.

Exactness for linear-Gaussian hierarchies. If $p(g)$ and every single-observation posterior $p(\vartheta_j \mid x_j)$ are Gaussian (as in a linear-Gaussian hierarchical model),

every backward kernel is proportional to a Gaussian density times the Gaussian forward kernel, and is therefore Gaussian. Equation (26) then holds with equality, and everything else in Steps 3 and 4 is exact algebra. So, given exact single-observation scores and covariances, Equations (37) and (38) give the exact tall-data score at every t.

Relation to the derivation of Linhart et al. [6]. Linhart et al. [6] obtain Equation (14) by differentiating the logarithm of the normaliser in Equation (13). That route produces a residual term, which vanishes when the covariances do not depend on the state. Going through the mean of the composed backward kernel (Equation (24)) instead, the same rule follows from Equation (26) alone, at one fixed y. The rule is exact whenever the product of the Gaussian surrogates has the same mean as the product of the true factors, and the covariances may vary with y. This is what licenses the state-dependent covariances of `gauss_jacobian`.

3.6 `gauss_hierarchical`: pilot covariances

`gauss_hierarchical` supplies the covariances in Equation (26) under the same model as GAUSS: each single-observation posterior over the *joint* block ϑ_j is taken to be Gaussian, $p(\vartheta_j \mid x_j) \approx \mathcal{N}(\mu_j, \Sigma_{0,j})$. For a Gaussian $p(\vartheta_0) = \mathcal{N}(\mu, \Sigma_0)$, the log backward kernel is $\log \pi_t(\vartheta_0 \mid y) = -\frac{1}{2}(\vartheta_0-\mu)^\top \Sigma_0^{-1}(\vartheta_0-\mu) - \frac{1}{2\lambda^2}\|y-\vartheta_0\|^2 + \text{const}$. The coefficient of its quadratic term in ϑ_0 is $-\frac{1}{2}(\Sigma_0^{-1} + \lambda^{-2}\mathbf{I})$, so the kernel is $\mathcal{N}\big(\cdot;\, m,\, (\Sigma_0^{-1} + \lambda^{-2}\mathbf{I})^{-1}\big)$ for some mean m. The diffused marginal $p_t = \mathcal{N}(\mu, \Sigma_0 + \lambda^2\mathbf{I})$ is the convolution of two Gaussians, with score $\nabla_y \log p_t(y) = -(\Sigma_0 + \lambda^2\mathbf{I})^{-1}(y-\mu)$. Applying this to each observation and to a Gaussian global prior gives

$$\begin{aligned} P_j^{\text{hier}} &= \Sigma_{0,j}^{-1} + \lambda^{-2}\mathbf{I}_D, \\ P_0 &= \Sigma_{\text{pr}}^{-1} + \lambda^{-2}\mathbf{I}_{d_g}, \\ s_0(g_t) &= -(\Sigma_{\text{pr}} + \lambda^2\mathbf{I})^{-1}(g_t - \mu_{\text{pr}}). \end{aligned} \tag{40}$$

$\Sigma_{0,j}$ is estimated once, before inference, as the empirical covariance of single-observation posterior draws from the same network (Section 4.2). Only the *covariance* is taken from the pilot. The means m_j in Equation (26) still come from the network scores via Equation (25), so the pilot mean μ_j never enters the rule. Substituting Equation (40) into Equation (34) and then into Equations (37) and (38) defines `gauss_hierarchical`.

`gauss_hierarchical` moves from marginal to conditional precision. Write $Q = \Sigma_{0,j}^{-1}$ with blocks $Q_{gg}, Q_{g\ell}, Q_{\ell\ell}$. Then

$$S_j = Q_{gg} + \lambda^{-2}\mathbf{I} - Q_{g\ell}\big(Q_{\ell\ell} + \lambda^{-2}\mathbf{I}\big)^{-1} Q_{\ell g}.$$

As $\lambda \to 0$, the last term is $\mathcal{O}(\lambda^2)$, so $S_j \simeq Q_{gg} + \lambda^{-2}\mathbf{I}$: the *conditional* precision of g given ℓ_j, plus the noise precision. As $\lambda \to \infty$, $\lambda^{-2}\mathbf{I} \to 0$ and $S_j \to Q_{gg} - Q_{g\ell}Q_{\ell\ell}^{-1}Q_{\ell g} = ([\Sigma_{0,j}]_{gg})^{-1}$, the *marginal* precision. Plain GAUSS on g alone (Section 2.6) uses $([\Sigma_{0,j}]_{gg})^{-1} + \lambda^{-2}\mathbf{I}$ at every noise level. It therefore agrees with `gauss_hierarchical` at high noise, uses the wrong weight at low noise, and never applies the local correction Equation (38).

3.7 `gauss_jacobian`: covariances from the network Jacobian

The Gaussian-posterior model behind `gauss_hierarchical` is a second approximation on top of Equation (26). If $p(\vartheta_j \mid x_j)$ is skewed or multimodal, $\Sigma_{0,j}^{-1} + \lambda^{-2}\mathbf{I}$ is not the backward precision at any t, and composition multiplies the resulting weighting error by n. `gauss_jacobian` removes this second approximation by using the exact covariance of each backward kernel, which the second-order Tweedie identity Equation (8) reads off the score Jacobian.

Second-order Tweedie. Take the setting of Equation (23) and assume additionally that p has a finite second moment. Write $\varphi = \varphi_\lambda(y-\theta_0)$ and $\delta = \theta_0 - y$. Differentiating $\nabla_y \varphi = \varphi\,\delta/\lambda^2$ once more, with $\nabla_y \delta^\top = -\mathbf{I}$, gives

$$\nabla_y^2 \varphi = \varphi\Big(\frac{\delta\delta^\top}{\lambda^4} - \frac{\mathbf{I}}{\lambda^2}\Big).$$

Differentiating under the integral sign (allowed as before, now using the second moment) and dividing by p_t,

$$\frac{\nabla_y^2 p_t(y)}{p_t(y)} = \frac{\mathbb{E}_{\pi_t}[\delta\delta^\top]}{\lambda^4} - \frac{\mathbf{I}}{\lambda^2}.$$

Combining this with $\nabla^2 \log p_t = \nabla^2 p_t / p_t - (\nabla \log p_t)(\nabla \log p_t)^\top$ and with Equation (23) in the form $\nabla \log p_t = \mathbb{E}_{\pi_t}[\delta]/\lambda^2$,

$$\begin{aligned} \nabla_y^2 \log p_t(y) &= \frac{\mathbb{E}_{\pi_t}[\delta\delta^\top] - \mathbb{E}_{\pi_t}[\delta]\mathbb{E}_{\pi_t}[\delta]^\top}{\lambda^4} - \frac{\mathbf{I}}{\lambda^2} \\ &= \frac{\mathrm{Cov}_{\pi_t}[\delta]}{\lambda^4} - \frac{\mathbf{I}}{\lambda^2}. \end{aligned}$$

Since δ and θ_0 differ only by the constant y, $\mathrm{Cov}_{\pi_t}[\delta] = \mathrm{Cov}_{\pi_t}[\theta_0 \mid y]$, and therefore

$$\nabla_y^2 \log p_t(y) = \frac{\mathrm{Cov}_{\pi_t}[\theta_0 \mid y] - \lambda^2\mathbf{I}}{\lambda^4}. \tag{41}$$

Covariances from the Jacobian. Let $J_j(y_j) := \nabla_{y_j} s_j(y_j) \in \mathbb{R}^{D\times D}$ be the Jacobian of observation j's joint score with respect to its own block. Rearranging Equation (41) gives the *exact* covariance of observation j's backward kernel and its inverse:

$$\begin{aligned} \Sigma_{t,j}^{\text{jac}}(y_j) &= \lambda^2\mathbf{I} + \lambda^4 J_j(y_j), \\ P_j^{\text{jac}}(y_j) &= \lambda^{-2}\big(\mathbf{I} + \lambda^2 J_j(y_j)\big)^{-1}. \end{aligned} \tag{42}$$

In the flat case this is the covariance of JAC. For an exact score, J_j is a Hessian, hence symmetric, and $\mathrm{Cov} \succeq 0$ implies $J_j \succeq -\lambda^{-2}\mathbf{I}$. With Equation (42), the Gaussians in Equation (26) are the moment-matched ones: they have the exact first moments (Equation (25)) and the exact second moments of the true factors. The blocks of Equation (42) are explicit, $[\Sigma_{t,j}]_{gg} = \lambda^2\mathbf{I} + \lambda^4 J_j^{gg}$ and $[\Sigma_{t,j}]_{\ell g} = \lambda^4 J_j^{\ell g}$, and Equation (34) only uses these blocks. So no $D \times D$ inversion is needed:

$$\begin{aligned} S_j^{\mathrm{jac}} &= \lambda^{-2}\big(\mathbf{I} + \lambda^2 J_j^{gg}\big)^{-1}, \\ R_j^{\mathrm{jac}} &= J_j^{\ell g}\big(\lambda^{-2}\mathbf{I} + J_j^{gg}\big)^{-1}. \end{aligned} \tag{43}$$

Substituting Equation (43) into Equations (37) and (38) defines `gauss_jacobian`.

`gauss_jacobian` contains `gauss_hierarchical` as a special case. If $p(\vartheta_j \mid x_j) = \mathcal{N}(\mu_j, \Sigma_{0,j})$ and the score is exact, then $J_j = -(\Sigma_{0,j} + \lambda^2\mathbf{I})^{-1}$, which does not depend on y. Since $\Sigma_{0,j}$ commutes with $(\Sigma_{0,j} + \lambda^2\mathbf{I})^{-1}$,

$$\begin{aligned} \Sigma_{t,j}^{\mathrm{jac}} &= \lambda^2\mathbf{I} - \lambda^4(\Sigma_{0,j} + \lambda^2\mathbf{I})^{-1} \\ &= \lambda^2(\Sigma_{0,j} + \lambda^2\mathbf{I})^{-1}\big[(\Sigma_{0,j} + \lambda^2\mathbf{I}) - \lambda^2\mathbf{I}\big] \\ &= \lambda^2(\Sigma_{0,j} + \lambda^2\mathbf{I})^{-1}\Sigma_{0,j}, \end{aligned}$$

and inverting gives

$$\begin{aligned} P_j^{\mathrm{jac}} &= \lambda^{-2}\Sigma_{0,j}^{-1}(\Sigma_{0,j} + \lambda^2\mathbf{I}) \\ &= \Sigma_{0,j}^{-1} + \lambda^{-2}\mathbf{I} = P_j^{\mathrm{hier}} \end{aligned} \tag{44}$$

for every y. `gauss_jacobian` thus keeps Equation (26) and drops the Gaussian-posterior model. On a Gaussian problem it matches `gauss_hierarchical` with the exact pilot covariance by construction, so a Gaussian benchmark cannot separate the two methods. They differ only when the single-observation posteriors are non-Gaussian. The approximation that remains is that the mean of a product of moment-matched Gaussians stands in for the mean of the product of the true factors. This is exact when the factors are Gaussian. It is least accurate where the single-observation backward kernels are strongly non-Gaussian (e.g. multimodal) at intermediate noise levels.

3.8 Summary: the two composition rules

Input at (t, y)**:** network row scores $s_j = (s_j^g, s_j^\ell)$; prior score s_0 and precision $P_0 = \Sigma_{\mathrm{pr}}^{-1} + \lambda^{-2}\mathbf{I}$.
Per-observation precision P_j**:**
`gauss_hierarchical`: $P_j = \hat{\Sigma}_{0,j}^{-1} + \lambda^{-2}\mathbf{I}$ (pilot)
`gauss_jacobian`: $P_j = \lambda^{-2}(\mathbf{I} + \lambda^2 J_j(y_j))^{-1}$ (Jacobian at y)
Elimination: $S_j = ([P_j^{-1}]_{gg})^{-1}$, $R_j = [P_j^{-1}]_{\ell g}[P_j^{-1}]_{gg}^{-1}$.
Composed score:

$$u^g = \Big[\textstyle\sum_j S_j + (1-n)P_0\Big]^{-1}\Big[\textstyle\sum_j S_j s_j^g + (1-n)P_0 s_0\Big],$$
$$u^{\ell_j} = s_j^\ell + R_j(u^g - s_j^g).$$

4 Implementation

Section 3 gives the rules for exact inputs. This section describes how the rules are evaluated with a trained network. It covers the quantities that are estimated, the safeguards that keep the linear algebra well posed, and the amortisation that keeps `gauss_jacobian` affordable. Every safeguard is inactive when its inputs satisfy the conditions of Section 3; we report how often each one fires. All rules are implemented in a single sampler class of our library; option names are given in typewriter font.

4.1 State layout and integration into a sampler

The sampler keeps an array of shape $n \times M \times D$ (observations × posterior draws × latent block). The global coordinates are synchronised across the observation axis, so row j holds $y_j = (g_t, \ell_{j,t})$. One batched network call evaluates all nM rows. The rows do not interact inside the network, so row j's output is $s_j(y_j)$ alone. The composition returns u^g on the global coordinates of every row and u^{ℓ_j} on row j's local coordinates, and the predictor (we use second-order DPM-Solver [7] with stochastic churn) uses this array as its score. All scores are converted to the variance-exploding coordinates $y = \theta_t/\alpha_t$ of Section 2.2 before composition. The composition linear algebra runs in float64.

4.2 Pilot covariances for `gauss_hierarchical`

Before inference, `gauss_hierarchical` draws P samples (default $P = 500$) from every single-observation posterior with the same network, in one batched reverse-diffusion run over all n observations. For each observation it forms the empirical covariance $\tilde{\Sigma}_{0,j}$ over the joint block ϑ_j and regularises it:

$$\hat{\Sigma}_{0,j} = (1-\rho)\,\tilde{\Sigma}_{0,j} + \rho\,\mathrm{diag}(\tilde{\Sigma}_{0,j}) + \epsilon\max\big(1, \tfrac{1}{D}\operatorname{tr}\tilde{\Sigma}_{0,j}\big)\mathbf{I}, \tag{45}$$

with shrinkage $\rho = 10^{-2}$ and nugget $\epsilon = 10^{-6}$. The pilot means are discarded, and the API rejects any externally supplied posterior mean (Section 3.6). Because P_j^{hier} depends only on λ, the factors (S_j, R_j) are cached per pair (λ, n).

4.3 Evaluating the Jacobian for `gauss_jacobian`

`gauss_jacobian` needs the $D \times D$ block $J_j = \nabla_{y_j} s_j$ for every row and draw. Rows do not interact, so a tangent that perturbs coordinate k in *every* row returns column k of *every* row's Jacobian. The full block therefore costs D directional derivatives, independent of n and of the data dimension. Perturbing a global coordinate in all rows at once is legitimate because each row reacts only to its own copy.

Forward mode. `gauss_jacobian` computes the D columns by forward-mode Jacobian-vector products. Fused scaled-dot-product attention kernels have no forward-mode rule, so the tangent passes, and only those, run attention through the reference (math) backend.

Finite differences. `gauss_jacobian_FD` uses forward differences, $J_j e_k \approx [s_j(y_j + he_k) - s_j(y_j)]/h$, with a step proportional to the noise level, $h = \kappa\lambda$ ($\kappa = 10^{-2}$). A λ-smoothed score varies on the length scale λ, which is why the step scales with it. The base state and the D perturbed copies are stacked along the draw axis and evaluated in a single no-grad network call, chunked to at most 16,384 rows. This keeps the fused attention kernel. In our measurements it is about 4× cheaper than D JVPs, with a relative Jacobian error of about 10^{-3} (up to $2 \cdot 10^{-2}$ at $\lambda \approx 3 \cdot 10^{-3}$), and the resulting posteriors agree with the JVP version to within Monte-Carlo noise. [**TODO:** Move these numbers into the experiments as an ablation.]

Architecture-specific shortcut. When the score network embeds every scalar parameter as its own token, never lets latent tokens attend to each other, and normalises per token, J_j is exactly diagonal. In that case a single all-ones tangent (or one finite-difference copy) gives the whole Jacobian. This does not hold for architectures whose latents attend to each other; we check it numerically before using it.

4.4 Symmetrisation and spectral clipping

The Jacobian of a trained network is not exactly symmetric. It is also not guaranteed to satisfy $J_j \succeq -\lambda^{-2}\mathbf{I}$, the bound that $\mathrm{Cov} \succeq 0$ imposes on a true score Jacobian (Section 3.7). We therefore form the symmetrised curvature $H_j = -\frac{1}{2}(J_j + J_j^\top)$, eigendecompose it as $H_j = V \operatorname{diag}(h_k) V^\top$, and clip its eigenvalues:

$$\begin{aligned} \tilde{h}_k &= \min\Big(\max(h_k, 0),\ \frac{1-f}{\lambda^2}\Big), \\ P_j &= V \operatorname{diag}\Big(\frac{1}{\lambda^2(1-\lambda^2\tilde{h}_k)}\Big) V^\top, \end{aligned} \tag{46}$$

with floor $f = 10^{-3}$. This is Equation (42) applied to the clipped spectrum, and it confines the precision to

$$\lambda^{-2}\mathbf{I} \preceq P_j \preceq (f\lambda^2)^{-1}\mathbf{I}.$$

The two clips have different status:

- The *upper* clip enforces a property of every true score Jacobian, $\mathrm{Cov}_{\pi_t} \succeq 0$, plus a margin f. It stops a locally degenerate Jacobian from claiming a point mass.
- The *lower* clip ($h_k \geq 0$, i.e. $\mathrm{Cov}_{\pi_t} \preceq \lambda^2\mathbf{I}$) is a modelling choice, not an identity. A backward kernel can be wider than the forward noise; this happens near the saddle between two posterior modes. The clip treats such directions as carrying no information beyond the noise.

In our experiments the lower clip is almost never active (0–2% of eigenvalues on trained networks, $< 0.1\%$ with exact scores), and disabling it alone changes no result. [**TODO:** Cite the ablation table.]

4.5 Amortising the Jacobian: refresh and caching

The curvature changes more slowly along the trajectory than the score does, so `gauss_jacobian` recomputes J_j only every k composed-score evaluations. What is cached between refreshes matters. P_j itself must not be reused at a later time. It contains an exact $\lambda^{-2}\mathbf{I}$ term, while the composed precision subtracts $(n-1)$ copies of $P_0(t) = \Sigma_{\mathrm{pr}}^{-1} + \lambda_t^{-2}\mathbf{I}$ built at the *current* time. With a stale P_j from $\lambda_{\mathrm{old}} > \lambda_t$, the noise terms of Λ_S sum to $n\lambda_{\mathrm{old}}^{-2} - (n-1)\lambda_t^{-2}$, which becomes negative as λ_t shrinks, so the solve breaks down partway along the schedule. We therefore cache the λ-free information

$$K_j := P_j - \lambda_{\mathrm{ref}}^{-2}\mathbf{I} \succeq 0 \tag{47}$$

($K_j \succeq 0$ by the lower bound of Section 4.4) and rebuild $P_j = K_j + \lambda_t^{-2}\mathbf{I}$ exactly at every evaluation. A lagged refresh then lags only the network's estimate of the clean information $\Sigma_{0,j}^{-1}$, which is the quantity `gauss_hierarchical` holds fixed for the whole trajectory. On a Gaussian problem $K_j = \Sigma_{0,j}^{-1}$ at every λ (Equation (44)), so every refresh period gives the same result.

The refresh period depends on the noise level, through the options `jacobian_refresh` (period at high noise), `jacobian_refresh_low_noise` (period at low noise) and `jacobian_refresh_lambda` (the threshold $\lambda^\star$). Lag is costly only at high noise. Our default refreshes at every evaluation above $\lambda^\star = 0.3$ and every 10 evaluations below it. This matched the JVP version at uniform refresh 20 to within 1–2% in posterior error at 0.62× its wall-clock time. [**TODO:** Move to the ablation.]

4.6 Elimination numerics

For each observation (and, for `gauss_jacobian`, each draw) we Cholesky-factorise P_j, form $\Sigma_{t,j} = P_j^{-1}$ by Cholesky solves, and read off $S_j = ([\Sigma_{t,j}]_{gg})^{-1}$ and $R_j = [\Sigma_{t,j}]_{\ell g} S_j$, as in Equation (34). For `gauss_jacobian` we could instead use the closed forms Equation (43) on the unclipped Jacobian. We do not, because the clipping of Section 4.4 acts on the full $D \times D$ spectrum, so the blocks are read from the clipped P_j. Large batched eigendecompositions (e.g. $n = 30$ observations × 3000 draws = 90,000 matrices) are chunked, because the batched GPU `eigh` workspace grows with the batch size.

4.7 Information projection

By Equation (39), $\Lambda_S = P_0 + \sum_j I_j$ with $I_j = S_j - P_0$, so $\Lambda_S \succ 0$ is guaranteed whenever each $I_j \succeq 0$. For flat Gaussians, $I_j = \Sigma_{0,j}^{-1} - \Sigma_{\text{pr}}^{-1} \succeq 0$ exactly when the single-observation posterior is no wider than the prior in any direction. By the law of total covariance this holds on average over x_j, but it can fail for individual observations, for example under prior–data conflict or with bimodal likelihoods. Estimation noise in S_j adds further violations. We project each I_j onto the positive semi-definite cone, which is its nearest point in Frobenius norm:

$$\begin{aligned} I_j &= \tfrac{1}{2}\big(I_j + I_j^\top\big) = U\,\mathrm{diag}(\iota_k)U^\top, \\ \Delta_j &= U\,\mathrm{diag}\big(\max(-\iota_k, 0)\big)U^\top, \end{aligned} \tag{48}$$

and add Δ_j to the global block A_j of P_j. Because $S_j = A_j - B_j C_j^{-1} B_j^\top$ is affine in A_j with unit slope, this raises S_j by exactly Δ_j, making $I_j \mapsto I_j^+$. It leaves $R_j = -C_j^{-1} B_j^\top$ unchanged, so the local regression is not disturbed. After projection, $\Lambda_S \succeq P_0$: the composed global precision is never weaker than the prior, and the correction does not grow with n. The projection is a no-op when every $I_j \succeq 0$. It is, however, often active. On a folded $|g|$ toy with exact scores it fires for 78–100% of information eigenvalues with `gauss_hierarchical` and about 18% with `gauss_jacobian`. Disabling it worsens the distance to the reference posterior in 42 of 148 experiment settings and improves it in 3. We keep it on by default (option `information_projection`) and report it as part of the method. [**TODO:** Ablation table.]

4.8 Composed-level repair

As a last resort against residual floating-point error, the composed matrix is checked before the solve. Let $\tau = \max(\epsilon_{\text{pd}}, 10^{-3}) \cdot \max(1, |\mathrm{eig}(\Lambda_S)|)$. If Λ_S fails a Cholesky factorisation or has an eigenvalue below τ, the deficient eigenvalues are lifted to τ by a correction Δ, and we solve

$$(\Lambda_S + \Delta)\, u^g = \textstyle\sum_j S_j s_j^g + (1-n) P_0 s_0 + \Delta\, \bar{s}^g,$$

where $\bar{s}^g = \frac{1}{n}\sum_j s_j^g$. In the repaired directions this shrinks the composed score towards the average row score instead of amplifying it by $1/\tau$. With the information projection on, $\Lambda_S \succeq P_0$ and the repair should not fire; every activation is logged. With the projection off, it fires in 45–68% of evaluations, so turning off the projection does not remove all safeguards.

4.9 Denoised-prediction clamps

Far in the tails of the training distribution the network extrapolates, and composition amplifies a single bad row score n-fold. Following common practice, we bound denoised predictions $\hat{\theta}_0 = y + \lambda^2 s$ to a box of $\pm k$ prior standard deviations ($k = 5$, option `denoise_clamp`) and convert back to a score with $s \leftarrow (\mathrm{clip}(\hat{\theta}_0) - y)/\lambda^2$. One composed-score evaluation proceeds as follows:

1. clamp every row's global prediction $g_t + \lambda^2 s_j^g$, and, if a marginal local prior is given, every row's local prediction;
2. build P_j (Section 4.2 or Sections 4.3 to 4.5), apply the information projection (Section 4.7), and form S_j, R_j;
3. solve for u^g (Equation (37), with the repair of Section 4.8);
4. clamp the composed global prediction $g_t + \lambda^2 u^g$;
5. apply the local rule Equation (38) using the *clamped* u^g, then clamp the local predictions again.

Step 5 uses the clamped u^g. Otherwise a poorly conditioned composition could reach the locals through $R_j(u^g - s_j^g)$ without passing through the only bound that the global coordinates have. The clamps are inactive in the bulk of the posterior. Where active, they make the composed field non-conservative, which matters for Langevin correctors [**TODO:** curl diagnostic].

4.10 Proposal priors

If the network was trained under a global prior $\tilde{p}(g)$ different from the target prior $p(g)$, then each factor is $\tilde{p}(\vartheta_j \mid x_j)$, and Equation (20) becomes $p(\theta \mid \mathbf{x}_{1:n}) \propto p(g) \prod_j \tilde{p}(\vartheta_j \mid x_j)/\tilde{p}(g)$. This holds because the likelihood does not depend on the prior. Repeating Section 3 with $\tilde{\pi}_t^0$ in place of π_t^0 for the divided-out priors changes only the prior terms:

$$\begin{aligned} \Lambda_S &= \textstyle\sum_j S_j - n\tilde{P}_0 + P_0, \\ \Lambda_S u^g &= \textstyle\sum_j S_j s_j^g - n\tilde{P}_0 \tilde{s}_0 + P_0 s_0, \end{aligned}$$

and the information of Section 4.7 is taken relative to $\tilde{P}_0$ (option `set_training_prior`).

4.11 Cost

Table 1 summarises the cost. `gauss_hierarchical` costs essentially the base trajectory plus the pilot. `gauss_jacobian` adds the Jacobian on top of the same trajectory, so it can approach the cost of `gauss_hierarchical` but cannot fall below it at equal step counts. On the partial-pooling benchmark, `gauss_jacobian` (FD, refresh 1/10) took 7,499 s, against 4,976 s for `gauss_hierarchical` and 10,018 s for F-NPSE with Langevin dynamics. [**TODO:** Move to experiments.]

5 Experiments

[**TODO:** Not yet written. Candidate experiments, each graded against a reference posterior:]

Table 1: Requirements and cost per composed-score evaluation, beyond the nM network row evaluations that every compositional sampler pays. $D = d_g + d_\ell$.

Rule	Pilot run	Extra per evaluation
Sum / F-NPSE	none	—
Flat GAUSS on g	P draws per obs.	$\mathcal{O}(nd_g^3)$
`gauss_hierarchical`	P draws per obs.	$\mathcal{O}(nD^3)$, cached in λ
`gauss_jacobian` (JVP)	none	D JVPs per refresh $+ \mathcal{O}(nMD^3)$
`gauss_jacobian` (FD)	none	D row copies per refresh $+ \mathcal{O}(nMD^3)$

- **Linear-Gaussian shared/local toy, exact scores.** Here both rules are exact (Section 3.5), so this isolates the composition rule from network error. Compare flat GAUSS applied to g against `gauss_hierarchical`/`gauss_jacobian`.
- **Non-Gaussian single-observation posteriors** (mixtures, folded $|g|$ likelihoods): `gauss_jacobian` versus `gauss_hierarchical` with exact moment-matched pilot covariances.
- **Partial-pooling cognitive model**, global/local accuracy and coverage against F-NPSE with Langevin and against the damped sum [1].
- **Inverse kinematics / hierarchical two moons**: multimodal stress tests.
- **Ablations**: information projection, curvature clip, finite-difference vs. JVP Jacobian, refresh schedule, and cost.

6 Conclusion

[**TODO:** Write last. Limitations to state: the Gaussian backward-kernel approximation (Equation (26)) is the only approximation in the rules, but it is not small when single-observation backward kernels are multimodal at intermediate noise levels; network Jacobians are not symmetric and are not exact Hessians; the safeguards of Section 4 are active in practice; and cost scales as $\mathcal{O}(D)$ extra network passes per Jacobian refresh.]

References

[1] Jonas Arruda et al. Compositional amortized inference for large-scale hierarchical Bayesian models. In *International Conference on Learning Representations*, 2026. arXiv:2505.14429. TODO: full author list.

[2] Tomas Geffner, George Papamakarios, and Andriy Mnih. Compositional score modeling for simulation-based inference. In *International Conference on Machine Learning*, 2023.

[3] Manuel Gloeckler, Michael Deistler, Christian Weilbach, Frank Wood, and Jakob H. Macke. All-in-one simulation-based inference. In *International Conference on Machine Learning*, 2024.

[4] Manuel Gloeckler et al. Compositional simulation-based inference for time series. In *International Conference on Learning Representations*, 2025. arXiv:2411.02728. TODO: full author list.

[5] Daniel Habermann, Marvin Schmitt, Lars Kühmichel, Andreas Bulling, Stefan T. Radev, and Paul-Christian Bürkner. Amortized Bayesian multilevel models. *arXiv preprint arXiv:2408.13230*, 2024.

[6] Julia Linhart, Gabriel Victorino Cardoso, Alexandre Gramfort, Sylvain Le Corff, and Pedro L. C. Rodrigues. Diffusion posterior sampling for simulation-based inference in tall data settings. *Transactions on Machine Learning Research*, 2026. arXiv:2404.07593.

[7] Cheng Lu, Yuhao Zhou, Fan Bao, Jianfei Chen, Chongxuan Li, and Jun Zhu. DPM-Solver: A fast ODE solver for diffusion probabilistic model sampling in around 10 steps. In *Advances in Neural Information Processing Systems*, 2022.

[8] Pedro L. C. Rodrigues, Thomas Moreau, Gilles Louppe, and Alexandre Gramfort. HNPE: Leveraging global parameters for neural posterior estimation. In *Advances in Neural Information Processing Systems*, 2021.

[9] Zaballa and Hui. TODO: exact title (hierarchical blockwise diffusion sampling, HBDS). *arXiv preprint arXiv:2609.36950*, 2026.